

LLM Training and Attention Design Tutorial / LLM 训练与 Attention 设计入门讲义

Chinese and English Printable Edition / 中英双语打印版

Author / 作者: liangdacheng / 梁达成

Printable PDF version



Contents

中文版本 / Chinese Version

1. 学习目标
2. 三阶段训练总览
3. Pre-train
4. SFT
5. Post-RL Training: DPO、RLHF、GRPO
6. Transformer 核心结构
7. Attention 设计代理指标
8. 关键公式
9. 推荐实验协议
10. 本地运行

English Version / 英文版本

1. Learning Goals
2. Three Training Stages
3. Pre-training
4. Supervised Fine-Tuning
5. Post-RL Training: DPO, RLHF, GRPO
6. Transformer Core Structures
7. Proxy Metrics for Attention Design
8. Key Formulas
9. Recommended Experiment Protocol
10. Local Execution

中文版本 / Chinese Version

以下为中文版本。

1. 学习目标

这份讲义把两个入门 notebook 合并成一份可打印材料：LLM 三阶段训练，以及 attention 设计指标。目标是先用小模型和可解释实验建立直觉，再迁移到真实模型、真实数据和分布式训练。

- 理解 pre-train、SFT、post-RL training 的目标、数据格式和 loss。
- 理解 embedding、attention、RoPE、residual、RMSNorm/LayerNorm、MLP 为什么存在。
- 掌握不完整训练 LLM 时评估 attention 设计的代理指标。
- 能用本地 conda 环境运行 PyTorch 和 Jupyter 入门实验。

2. 三阶段训练总览



Pre-train 教模型学习文本分布；SFT 教模型按照指令格式回答；post-RL training 用偏好、奖励或验证器进一步对齐输出行为。

阶段	数据	核心 loss / 方法	输出模型
Pre-train	大规模原始文本	causal LM next-token cross entropy	Base model
SFT	指令、问题、标准答案	assistant token masked cross entropy	Instruction model
DPO	prompt + chosen + rejected	relative preference objective with reference model	Preference-aligned model
GRPO/RLHF	prompt + reward/verifier 或偏好数据	policy optimization / reward model	Aligned model

3. Pre-train

Pre-train 的基本目标是预测下一个 token。输入和标签几乎相同，只是在 loss 里 shift 一个位置。

```
input:  x0 x1 x2 ... xT
target:   x1 x2 ... xT
loss:    CE(model(x_{<=t}), x_{t+1})
```

真实 pre-train 的难点通常不在公式，而在数据工程、tokenizer、吞吐、稳定性、checkpoint、验证集污染和规模化训练。入门时建议从 100M 以内 tiny GPT 或 0.5B 级 continued pretraining 开始。

4. SFT

SFT 仍然是 causal LM loss，但只应该监督 assistant 需要生成的 tokens。user prompt 是条件输入，不是模仿目标。

```
input:  <bos>User: 如何 SFT?\nAssistant: SFT ...
labels: -100 -100 -100 ...           SFT ...
```

- chat template 必须和模型/tokenizer 匹配。
- 长回答要注意截断策略，否则 assistant 监督信号会被截掉。
- 高质量小数据通常胜过低质量大数据。
- 评估不能只看训练 loss，还要看格式遵循、事实性、拒答、任务集和人工样例。

5. Post-RL Training: DPO、RLHF、GRPO

Post-RL training 的目的不是重新教语言，而是让模型输出更符合偏好、规则、奖励或可验证目标。DPO 是最适合入门的 post-RL 方法，因为它不需要单独训练 reward model。

$$L_{DPO} = -\log \text{sigmoid}(\beta * ((\log \pi_{\text{chosen}} - \log \pi_{\text{rejected}}) - (\log \pi_{\text{ref_chosen}} - \log \pi_{\text{ref_rejected}})))$$

方法	需要的数据	适合场景
DPO	chosen/rejected 偏好对	SFT 后的偏好对齐入门
Reward Model + PPO	偏好对 + RL 采样	传统 RLHF 管线
GRPO	prompt + 可验证 reward function	数学、代码、规则可验证任务
RLAIF	AI 反馈或规则生成偏好	人类标注不足时的扩展方案

6. Transformer 核心结构



结构	解决的问题	去掉后的常见问题	改进方向
Embedding	离散 token 到连续向量	token id 没有语义距离	tokenizer、weight tying、domain vocabulary
Attention	上下文信息路由	长距离依赖弱	FlashAttention、GQA/MQA、sparse/window attention
RoPE/位置编码	注入顺序和相对距离	顺序、括号、代码位置能力差	YaRN、LongRoPE、ALiBi、xPos
Residual	深层信息和梯度通路	深层训练不稳定	residual scaling、DeepNorm、gated residual
RMSNorm/LayerNorm	稳定激活分布	loss spike、梯度爆炸/漂移	Pre-Norm、QK-Norm、RMSNorm
MLP/SwiGLU	非线性容量和知识存储	表达力不足	SwiGLU、MoE、扩展 ratio

7. Attention 设计代理指标

不完整训练 LLM 时，可以用 proxy 指标提前筛掉风险设计。单个指标不能替代最终 loss，但组合后很有价值。

维度	指标	主要预测
信息流	graph diameter, receptive field, rollout mass	长距离信息是否可传递
稳定性	QK logits std/p99/p99.9, normalized entropy	attention 是否饱和或训练崩
表达力	effective rank, head diversity, ablation	attention 是否退化或 head 浪费
位置能力	retrieval vs distance, extrapolation ratio	长上下文和长度外推

硬件	tokens/sec, TTFT, TPOT, MFU, KV cache	实际训练和部署可用性
小任务	copy, key-value retrieval, multi-hop	结构能力下限

8. 关键公式

```

Receptive field:
 $R_0(t) = \{t\}$ 
 $R_l(t) = \text{union over } s \text{ in } \text{Attend}_l(t) \text{ of } R_{l-1}(s)$ 

Attention logits:
 $z_{\{t,s\}} = q_t \cdot k_s / \sqrt{d_{\text{head}}}$ 
 $p_{\{t,s\}} = \text{softmax}(z_{\{t,s\}})$ 

Normalized entropy:
 $H_t = - \sum_s p_{\{t,s\}} \log p_{\{t,s\}}$ 
 $H_{\text{norm}} = H_t / \log(N_t)$ 

Effective rank:
 $p_i = \sigma_i / \sum_j \sigma_j$ 
 $r_{\text{eff}} = \exp(- \sum_i p_i \log p_i)$ 

KV cache:
KV bytes = layers * batch * seq_len * 2 * n_kv_heads * head_dim * dtype_bytes

```

9. 推荐实验协议

- 先静态计算 FLOPs、KV cache、参数量。
- 随机输入下统计 QK logits、entropy、effective rank、head diversity。
- 短训 1k-5k steps, 观察 loss spike、NaN/Inf、logits 漂移。
- 跑 copy、key-value retrieval、multi-hop 小任务。
- 做 retrieval vs distance, 观察 lost-in-the-middle 和长度外推。
- 最后再做小规模 LLM pre-train loss 和真实任务评估。

10. 本地运行

当前工作目录已经包含 conda 环境文件、Jupyter 启动脚本和两个 notebook。

```

cd /mnt/c/Users/Administrator/Desktop/
./verify_llm_intro_env.sh
./start_llm_intro_jupyter.sh

```

文件	用途
llm_training_stages/llm_pretrain_sft_postrl_tutorial.ipynb	Pre-train、SFT、DPO 教学实验
attention_design_metrics/attention_metrics_tutorial.ipynb	Attention 设计指标教学实验
llm_training_stages/tiny_llm_training.py	tiny GPT、SFT batch、DPO loss
attention_design_metrics/attention_metrics_to_rch.py	attention 指标工具函数
environment.yml	conda 环境定义

English Version / 英文版本

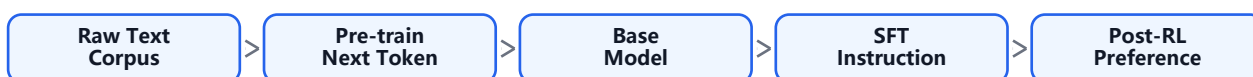
The following section is the English version.

1. Learning Goals

This handout merges two introductory notebooks into one printable guide: LLM training stages and attention-design metrics. The goal is to build intuition with small, inspectable experiments before scaling to real models, real datasets, and distributed training.

- Understand the goal, data format, and loss for pre-training, SFT, and post-RL training.
- Understand why embedding, attention, RoPE, residual paths, RMSNorm/LayerNorm, and MLP blocks exist.
- Use proxy metrics to evaluate attention designs before running full LLM training.
- Run the local PyTorch and Jupyter intro experiments from the provided conda environment.

2. Three Training Stages



Pre-training teaches the model a text distribution. SFT teaches the model to follow instructions and answer in the desired format. Post-RL training further aligns behavior with preferences, rewards, or verifiable objectives.

Stage	Data	Core loss / method	Output model
Pre-train	Large raw text corpora	causal LM next-token cross entropy	Base model
SFT	Instructions, questions, reference answers	assistant-token masked cross entropy	Instruction model
DPO	prompt + chosen + rejected	relative preference objective with reference model	Preference-aligned model
GRPO/RLHF	prompts + rewards/verifiers or preferences	policy optimization / reward model	Aligned model

3. Pre-training

The basic objective is next-token prediction. Inputs and labels are almost identical, but the loss is shifted by one token.

```
input:  x0 x1 x2 ... xT
target:   x1 x2 ... xT
loss:   CE(model(x_{<=t}), x_{t+1})
```

In real pre-training, the hard parts are usually not the formula. The hard parts are data quality, tokenization, throughput, stability, checkpointing, contamination control, and scale-out training. For learning, start with a tiny GPT below 100M parameters or a 0.5B-class continued pre-training run.

4. Supervised Fine-Tuning

SFT is still causal LM training, but labels should supervise only the assistant tokens. The user prompt is conditioning context, not a target to imitate.

```
input: <bos>User: What is SFT?\nAssistant: SFT is...
labels: -100 -100 -100 ...           SFT is...
```

- The chat template must match the model and tokenizer.
- Watch truncation: it can remove the assistant tokens that carry the training signal.
- A small high-quality dataset often beats a large noisy one.
- Do not evaluate only training loss; also check format following, factuality, refusals, task suites, and generated samples.

5. Post-RL Training: DPO, RLHF, GRPO

Post-RL training is not meant to relearn language from scratch. It aligns output behavior with preferences, rules, rewards, or verifiable targets. DPO is the best entry point because it does not require a separate reward model.

```
L_DPO = -log sigmoid(beta * ((log pi_chosen - log pi_rejected)
                             - (log ref_chosen - log ref_rejected)))
```

Method	Required data	Best use case
DPO	chosen/rejected preference pairs	Intro alignment after SFT
Reward Model + PPO	preference pairs + RL sampling	Classic RLHF pipeline
GRPO	prompts + verifiable reward function	Math, code, and rule-verifiable tasks
RLAIF	AI feedback or rule-generated preferences	When human labels are scarce

6. Transformer Core Structures



Structure	Problem solved	Common issue if removed	Improvement directions
Embedding	Map discrete tokens to vectors	Token IDs carry no semantic distance	tokenizer, weight tying, domain vocabulary
Attention	Route context information	Weak long-range dependency handling	FlashAttention, GQA/MQA, sparse/window attention
RoPE/position encoding	Inject order and relative distance	Poor sequence order, brackets, and code position handling	YaRN, LongRoPE, ALiBi, xPos
Residual path	Information and gradient highway	Deep training becomes unstable	residual scaling, DeepNorm, gated residual
RMSNorm/LayerNorm	Stabilize activation distribution	loss spikes, exploding gradients, drift	Pre-Norm, QK-Norm, RMSNorm
MLP/SwiGLU	Nonlinear capacity and knowledge storage	Insufficient expressivity	SwiGLU, MoE, larger expansion ratio

7. Proxy Metrics for Attention Design

Before running full LLM training, proxy metrics can eliminate risky designs. No single metric replaces final loss, but the combination is useful.

Dimension	Metrics	Main prediction
Information flow	graph diameter, receptive field, rollout mass	Whether long-range information can travel
Stability	QK logits std/p99/p99.9, normalized entropy	Whether attention saturates or training breaks
Expressivity	effective rank, head diversity, ablation	Whether attention collapses or heads are wasted
Positional ability	retrieval vs distance, extrapolation ratio	Long-context and length extrapolation
Hardware	tokens/sec, TTFT, TPOT, MFU, KV cache	Practical training and serving viability
Small tasks	copy, key-value retrieval, multi-hop	Lower bound of structural capability

8. Key Formulas

Receptive field:

$$R_0(t) = \{t\}$$
$$R_l(t) = \text{union over } s \text{ in } \text{Attend}_l(t) \text{ of } R_{l-1}(s)$$

Attention logits:

$$z_{\{t,s\}} = q_t \cdot k_s / \sqrt{d_{\text{head}}}$$
$$p_{\{t,s\}} = \text{softmax}(z_{\{t,s\}})$$

Normalized entropy:

$$H_t = - \sum_s p_{\{t,s\}} \log p_{\{t,s\}}$$
$$H_{\text{norm}} = H_t / \log(N_t)$$

Effective rank:

$$p_i = \sigma_i / \sum_j \sigma_j$$
$$r_{\text{eff}} = \exp(- \sum_i p_i \log p_i)$$

KV cache:

$$\text{KV bytes} = \text{layers} * \text{batch} * \text{seq_len} * 2 * n_{\text{kv_heads}} * \text{head_dim} * \text{dtype_bytes}$$

9. Recommended Experiment Protocol

- First compute FLOPs, KV cache, and parameter count statically.
- On random inputs, inspect QK logits, entropy, effective rank, and head diversity.
- Train for 1k-5k steps and watch for loss spikes, NaN/Inf, and logit drift.
- Run copy, key-value retrieval, and multi-hop synthetic tasks.
- Measure retrieval vs distance, including lost-in-the-middle and length extrapolation.
- Only then run small-scale LLM pre-training loss and real task evaluation.

10. Local Execution

The working directory already contains the conda environment file, Jupyter startup script, and both notebooks.

```
cd /mnt/c/Users/Administrator/Desktop/□□□□
```

```
./verify_llm_intro_env.sh
./start_llm_intro_jupyter.sh
```

File	Purpose
llm_training_stages/llm_pretrain_sft_postrl_tutorial.ipynb	Pre-train, SFT, and DPO teaching experiment
attention_design_metrics/attention_metrics_tutorial.ipynb	Attention-design metric teaching experiment
llm_training_stages/tiny_llm_training.py	tiny GPT, SFT batch, DPO loss
attention_design_metrics/attention_metrics_to_rch.py	attention metric utilities
environment.yml	conda environment definition